Code walkthrough

FSO channel with weak turbulence and pointing errors — Python implementation

Sweety Kumari Vishwakarma

October 31, 2026

This document explains the Python code delivered with the assignment, module by module and function by function: what each piece computes, which equation of the paper it implements, and why it is written the way it is. The code has four files:

File	Role
`fso_model.py`	parameters (Table I), operating points, random samplers, PDFs (5), (7), (10), (14)
`ber.py`	BER (15), exact average BER (16), approximations (17)/(18), area, outage, remedy
`montecarlo.py`	bit-level Monte Carlo of the system in Fig. 1 and block-fading outage MC
`run_all.py`	runs every task, makes all figures and `results/results.json`

Contents

1 Overall structure and data flow **1**

2 `fso_model.py` **2**
- 2.1 `SystemParams`: Table I and derived quantities . . . 2
- 2.2 `OperatingPoint`: one (σ_s, σ_R^2) pair . . . 2
- 2.3 Samplers . . . 3
- 2.4 PDFs . . . 3

3 `ber.py` **4**
- 3.1 Conditional BER, eq. (15) . . . 4
- 3.2 Exact average BER, eq. (16) . . . 4
- 3.3 Approximations (17), (18) and the area under (14) . . . 4
- 3.4 Bounded-Q remedy . . . 4
- 3.5 Outage (stretch b) and Δ . . . 5

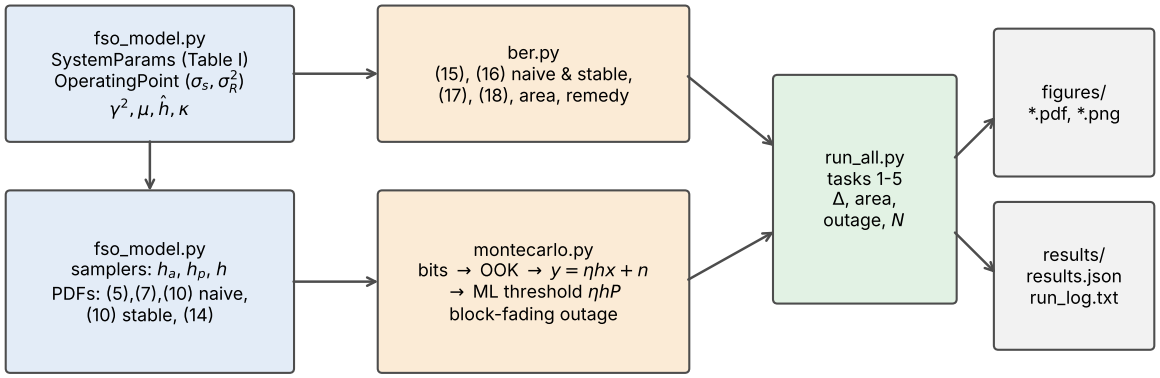
4 `montecarlo.py` **5**

5 `run_all.py` **5**

6 Checks built into the results **6**

7 How to run **6**

1 Overall structure and data flow



The model file knows nothing about BER; the BER file knows nothing about Monte Carlo; `run_all.py` glues them together. Every BER curve is computed twice by independent routes (numerical integration in `ber.py` and bit-level simulation in `montecarlo.py`), so an error in either route shows up as a disagreement in the figures.

The one change of variable used everywhere

All integrals use the standardised log-gain

$$u = \ln \frac{h}{A_0 h_l}, \qquad z = \frac{u + 2\sigma_X^2}{2\sigma_X}, \qquad \kappa = 2\gamma^2 \sigma_X = \gamma^2 \sigma_R.$$

In this variable turbulence contributes a standard Gaussian W and the pointing error an independent exponential $E' \sim \mathrm{Exp}(\kappa)$, so $z = W - E'$ and the exact PDF (10) is

$$f_Z(z) = \kappa\, e^{\kappa z + \kappa^2/2}\, Q(z + \kappa),$$

while the paper's approximation (14) becomes $\kappa\phi(z)/(z+\kappa)$ for $z > -\kappa$ (derivations in the report, Sec. 1). The reason to work in z is numerical: z is $O(1)$ for all operating points, whereas h is around 10^{-3} and the factors in (10) range over hundreds of orders of magnitude.

2 `fso_model.py`

2.1 `SystemParams`: Table I and derived quantities

A frozen dataclass holds the seven numbers of Table I. Everything else is a `@property`, so it is always consistent with the inputs:

```
@property
def wz(self): # beam waist at receiver, w_z ~ theta_1/2 * L
    return self.theta_half * self.L
@property
def v(self): # v = sqrt(pi) a / (sqrt(2) w_z)
    return np.sqrt(np.pi) * self.a / (np.sqrt(2) * self.wz)
@property
def A0(self): # collected fraction without pointing error
    return sp.erf(self.v) ** 2
@property
def wzeq2(self): # equivalent beam width squared
    v = self.v
    return self.wz**2 * np.sqrt(np.pi) * sp.erf(v) / (2 * v * np.exp(-v**2))
@property
def hl(self): # Beer-Lambert, sigma given in dB/km
    return 10 ** (-self.att_dB_per_km * (self.L / 1e3) / 10)
```

Numbers produced: $w_z = 1.98$ m, $v = 0.0316$, $A_0 = 1.275 \times 10^{-3}$, $w_{z,eq} = 1.981$ m, $h_l = 0.8585$. Table I gives the attenuation in dB/km, so eq. (9) is applied as $h_l = 10^{-0.2208 \cdot 3/10}$; this only shifts all curves by the same 0.66 dB and does not affect Δ. `rytov_from_Cn2` / `Cn2_from_rytov` implement eq. (4).

2.2 `OperatingPoint`: one (σ_s, σ_R^2) pair

```
@property
def sX2(self): return self.sigma_R2 / 4 # eq. (3)
@property
def gamma(self): return self.p.wzeq / (2 * self.sigma_s) # eq. (8)
@property
def mu(self): return 2 * self.sX2 * (1 + 2 * self.g2) # eq. (11)
@property
def hhat(self): return self.A0hl * np.exp(-self.mu) # support of (14)
@property
def kappa(self): return 2 * self.g2 * self.sX
@property
def big_exponent(self): # argument of exp() in (10)
    return 2 * self.sX2 * self.g2 * (1 + self.g2)
def h_from_z(self, z): return self.A0hl * np.exp(2 * self.sX * z - 2 * self.sX2)
def z_from_h(self, h): return (np.log(h / self.A0hl) + 2 * self.sX2) / (2 * self.sX)
```

`big_exponent` is printed and plotted because it is exactly what overflows: double precision holds e^x only for $x < 709.78$. For the green point it equals 2353.

2.3 Samplers

```
def sample_ha(n, op, rng):
    X = rng.normal(-op.sX2, op.sX, n) # log-amplitude, mean -sX2 => E[h_a]=1
    return np.exp(2 * X) # intensity = exp(2 X), eq. (5)

def sample_hp(n, op, rng):
    x = rng.normal(0.0, op.sigma_s, n) # horizontal jitter
    y = rng.normal(0.0, op.sigma_s, n) # vertical jitter
    R2 = x * x + y * y # R is Rayleigh
    return op.p.A0 * np.exp(-2 * R2 / op.p.wzeq2) # mapped through eq. (6)

def sample_h(n, op, rng):
    return op.p.hl * sample_hp(n, op, rng) * sample_ha(n, op, rng) # eq. (2)
```

The samplers follow the physics, not the PDFs: turbulence is drawn as a Gaussian log-amplitude, and the pointing error as two Gaussian displacements whose radius is pushed through the Gaussian-beam collection formula (6). The PDFs (5), (7) and (10) are *never* used to generate samples, so comparing the histograms with the PDFs (Fig. `fig_model_check`) is a genuine check of the derivations.

2.4 PDFs

`pdf_H_naive` codes eq. (10) literally, as a first implementation would:

```
return (g2 / (2 * A0hl**g2) * h**(g2 - 1)
       * sp.erfc((np.log(h / A0hl) + mu) / np.sqrt(8 * s))
       * np.exp(2 * s * g2 * (1 + g2)))
```

For the green point `np.exp(2353)` returns `inf` and `erfc` of an argument near 48 returns `0`, so the product is `nan`. It is kept deliberately to reproduce the failure the paper describes.

`logpdf_Z` / `pdf_H_stable` evaluate the same function without ever forming the huge and tiny factors separately:

```
def logpdf_Z(z, kappa):
    return np.log(kappa) + kappa*z + 0.5*kappa**2 + sp.log_ndtr(-(z + kappa))
def pdf_H_stable(h, op):
    z = op.z_from_h(h)
    return np.exp(logpdf_Z(z, op.kappa)) / (2 * op.sX * h) # Jacobian dz/dh
```

`scipy.special.log_ndtr(x)` returns $\ln \Phi(x)$ accurately even when $\Phi(x)$ is far below the smallest double, so $\ln Q(z+\kappa) = \texttt{log_ndtr}(-(z+\kappa)) \approx -\kappa^2/2$ cancels $+\kappa^2/2$ *in the log domain*. Only the final, moderate number is exponentiated. Wherever the naive formula works it gives the same average BER as the stable one (yellow: identical to 10^{-6} at all powers; pink: identical down to $\bar{P}_b \approx 10^{-8}$, below which `quad`'s absolute tolerance, not the PDF, limits the naive integral).

`cdf_Z` is the closed-form CDF $F_Z(z) = \Phi(z) + e^{\kappa z + \kappa^2/2} Q(z+\kappa)$, again in log form. It gives the outage probability with no integration at all.

`pdf_Z_approx` / `pdf_H_approx` implement eq. (14) after merging all exponents (report, Sec. 1.4):

```
out = kappa * np.exp(-0.5 * z**2) / np.sqrt(2*np.pi) / (z + kappa)
return np.where(z > -kappa, out, 0.0)
```

`pdf_H_approx_as_written` codes (14) term by term as printed; it overflows exactly like (10), which shows that (14) is only stable once the exponents are combined.

3 `ber.py`

3.1 Conditional BER, eq. (15)

```
def cond_ber(h, P_W, op):
    return 0.5 * sp.erfc(op.p.eta * P_W * h / (np.sqrt(2) * op.p.sigma_n))
```

3.2 Exact average BER, eq. (16)

```
def _z_grid_exact(kappa, n=40001):
    zmin = -kappa - 50.0 / kappa - 12.0 # left tail decays like exp(kappa z)
    zmax = 12.0 # right tail is Gaussian
    return np.linspace(zmin, zmax, n)

def ber_exact(P_dBm, op, n=40001):
    z = _z_grid_exact(op.kappa, n)
    w = np.exp(logpdf_Z(z, op.kappa))
    h = op.h_from_z(z)
    return np.array([integrate.simpson(cond_ber(h, P, op) * w, x=z)
                     for P in np.atleast_1d(dBm_to_W(P_dBm))])
```

The integral $\int P_b(h) f_H(h)\, dh$ is rewritten as $\int P_b(h(z)) f_Z(z)\, dz$ and done with Simpson's rule on 40 001 points. The grid limits come from the tails of f_Z: on the left it decays like $e^{\kappa z}$ (deep pointing fades), so $50/\kappa$ extra units leave e^{-50} of the mass outside; on the right it decays like a Gaussian. Increasing n fourfold changes the results by less than 2×10^{-14} (relative).

`ber_exact_naive` is the "off-the-shelf" version: `scipy.integrate.quad` over h with `pdf_H_naive`. It returns `nan` for the green point at every power.

3.3 Approximations (17), (18) and the area under (14)

Eq. (14) contains $1/(z+\kappa)$, inherited from $\operatorname{erfc}(x) \approx e^{-x^2}/(x\sqrt{\pi})$, which is infinite at $x = 0$, i.e. at $h = \hat{h}$. The substitution $t = \ln(z+\kappa)$ gives $dz/(z+\kappa) = dt$, so the integrand becomes smooth:

```
def _t_grid(kappa, delta, n=40001, zmax=12.0):
    return np.linspace(np.log(delta), np.log(zmax + kappa), n)

def ber_approx17(P_dBm, op, delta=1e-6, n=40001):
    t = _t_grid(op.kappa, delta, n)
    z = np.exp(t) - op.kappa
    phi = np.exp(-0.5 * z * z) / np.sqrt(2 * np.pi)
    h = op.h_from_z(z)
    return np.array([integrate.simpson(op.kappa * phi * cond_ber(h, P, op), x=t)
                     for P in ...])
```

As $t \to -\infty$ the integrand tends to the constant $\kappa\phi(\kappa)P_b(\hat{h})$, so the integral grows like $\ln(1/\delta)$: (17) and the area under (14) are, strictly, divergent. The code therefore takes the lower limit $z = -\kappa + \delta$ as an explicit argument, uses $\delta = 10^{-6}$ by default, and reports results for several δ. `divergence_rate(op)` returns $\kappa\phi(\kappa)$, the extra area per e-fold of δ; it is 4×10^{-2} for the pink point and at most 10^{-5} for all other points. `ber_approx18` is identical except that $P_b(h)$ is also replaced by its asymptotic form. `lost_mass(op)` returns $F_Z(-\kappa)$, the exact probability below $\hat{h}$ that (14) discards.

3.4 Bounded-Q remedy

`pdf_Z_remedy` replaces $Q(y) \approx \phi(y)/y$ by the Börjesson–Sundberg form $Q(y) \approx \phi(y)/((1-a)y + a\sqrt{y^2+b})$ with $a = 0.339$, $b = 5.51$ (the paper's choice is $a = 0$), and uses $Q(y) = 1 - Q(-y)$ for $y < 0$. The PDF is then finite at $\hat{h}$ and defined for all h; `area_remedy` and `ber_remedy` are used in the report to show that the slope problem of (17) disappears.

3.5 Outage (stretch b) and Δ

```
def h_threshold(P_dBm, op, target=HD_FEC): # P_b(h_th) = target
    return np.sqrt(2) * op.p.sigma_n * sp.erfcinv(2*target) / (op.p.eta * dBm_to_W(P_dBm))
def outage_prob(P_dBm, op, target=HD_FEC):
    return cdf_Z(op.z_from_h(h_threshold(P_dBm, op, target)), op.kappa)
```

Because $P_b(h)$ decreases with h, "$P_b(h) >$ target" is the same event as "$h < h_{\text{th}}$", and its probability is the closed-form CDF. `crossing_dBm` interpolates $\log_{10} \bar{P}_b$ linearly in P[dBm] to find where a curve meets 3.84×10^{-3}; Δ is the difference of two crossings. The power grid is 0.05–0.1 dB, so interpolation error is below 10^{-3} dB.

4 `montecarlo.py`

```
b = rng.integers(0, 2, m) # information bits
x = 2 * P * b # OOK symbols {0, 2P}
h = sample_h(m, op, rng) # fading, eq. (2)
y = eta * h * x + sn * rng.standard_normal(m) # eq. (1)
b_hat = (y > eta * h * P).astype(b.dtype) # ML hard decision with CSI
n_err += int(np.count_nonzero(b_hat != b))
```

- **Interleaver.** The paper assumes an infinitely long interleaver, which makes the fading seen by consecutive symbols independent. Drawing a new h for each bit is exactly that channel, so Π and Π^{-1} need not be simulated explicitly.
- **Detector.** With equiprobable bits and known h, the two hypotheses are $\mathcal{N}(0, \sigma_n^2)$ and $\mathcal{N}(2\eta hP, \sigma_n^2)$; the ML rule is the midpoint threshold ηhP. Its error probability is $Q(\eta hP/\sigma_n)$ = eq. (15).
- **Size.** 4×10^6 bits per power point in chunks of 10^6 (memory stays small). At the HD-FEC limit this gives about 1.5×10^4 errors, i.e. a relative standard error of about 0.8 %, which

corresponds to less than 0.01 dB on the power axis. Powers whose exact BER is below 3×10^{-6} are skipped.

- **Reproducibility.** One `numpy.random.Generator` seeded with 2024 is passed to every function.

`simulate_outage` draws one h per coherence block and counts the blocks whose conditional BER exceeds the target; it checks the closed-form outage.

5 `run_all.py`

Function	Task	Output
`fig_model_check`	1	MC histograms of h_a, h_p, z vs (5), (7), (10), (14)
`nine_points`	3	area under (14), Δ_{17}, Δ_{18}, lost mass, remedy, 9 points
`fig_instability`	2, 3	where naive (10) fails, κ contours, Δ at 9 points
`ber_curves`, `fig_ber`	2	Fig. 2 (right) reproduction, per-point panels
`fig_area`	3	area vs δ; Δ vs κ
`outage_study`	4(b)	outage vs average BER, power penalties
`interleaver_study`	5	required power vs interleaver depth N

`naive_fails(op)` evaluates naive (10) on 400 points covering the bulk of the distribution and flags the operating point if any value is non-finite or differs from the stable value by more than 10^{-6} (relative) where the true density is non-negligible. Sweeping it over a 131×119 grid of (σ_s, σ_R^2) gives the red region of `fig_instability`; the dashed line is the analytic boundary $2\sigma_X^2\gamma^2(1+\gamma^2) = 709.78$.

`required_power(op, N, ...)` draws $M \times N$ gains (one row per codeword, N independent fades per codeword), averages the conditional BER over each row, and bisects on P until the fraction of rows above the HD-FEC limit equals $q = 10^{-3}$. The same samples are reused for every P (common random numbers), which makes the bisection monotone and smooth. $N = 1$ is checked against the closed-form outage, and $N \to \infty$ is the average-BER answer.

6 Checks built into the results

1. Histograms of samples generated from the physics match (5), (7) and the stable (10).
2. Stable (16) equals naive (16) wherever the naive one works (yellow; pink down to $\bar{P}_b \approx 10^{-8}$).
3. Bit-level Monte Carlo agrees with stable (16) at all three points, including the green point where (16) cannot be evaluated naively: 40 of 41 simulated powers lie within 2.7 standard errors; the one outlier (3.6σ, green, 4 dBm) was re-run with three other seeds and 8×10^6 bits and landed within 1σ.
4. Closed-form outage agrees with block-fading Monte Carlo; $N = 1$ of the interleaver study agrees with closed-form outage.
5. Δ_{17} and Δ_{18} agree with the values printed in the paper at eight of nine points; the ninth (pink) is explained by the divergence of (17).

7 How to run

```
pip install numpy scipy matplotlib
cd code
python run_all.py # full run, 10-15 minutes on a laptop
python run_all.py --quick # 10x fewer Monte Carlo bits
```

Figures are written to `figures/` (PDF and PNG) and every number quoted in the report and slides to `results/results.json`; the console log is in `results/run_log.txt`.